

Document 2: Updated Installation Document

Submission Document

Solution Name: **Quantum Shield: Enterprise Post-Quantum Security Platform**

Repository: <https://github.com/Mohitlikestocode/Quantum-Proof-Systems-Scanner> PNB

Generated: 2026-07-18 10:19

1. Installation Scope

This guide provisions Quantum Shield end-to-end for development, technical demonstration, and deployment validation. It includes full hardware/software requirements, operating-system compatibility, dependency inventory, runtime configuration, startup commands, verification checks, and hardening checkpoints.

2. Hardware Sizing

Profile	CPU	Memory	Disk	Network
Minimum (Local Demo)	2 logical cores	4 GB RAM	2 GB free	Stable internet
Recommended (Team Dev)	4-8 logical cores	8-16 GB RAM	5-10 GB free	Broadband
Staging / Internal Pilot	8+ logical cores	16 GB+ RAM	20 GB+ SSD	Low-latency enterprise network

3. Supported Operating Systems

Operating System	Support	Notes
Windows 10/11 (x64)	Primary	Validated for PowerShell workflow and backend report generation
Ubuntu 20.04+ / Debian-based Linux	Supported	Use python3 + pip3 and Node.js LTS packages
macOS 12+ (Apple Silicon / Intel)	Supported	Use Homebrew-managed Python and Node.js

4. Mandatory Software Prerequisites

Component	Version	Purpose
Python	3.10+	Backend API, engines, scheduler, PDF generation
pip	23+ recommended	Install backend libraries from requirements.txt
Node.js	20.19+ or 22.12+	Frontend runtime and Vite tooling
npm	10+ recommended	Install JavaScript dependencies and run scripts
Git	2.35+	Source retrieval and version tracking

Optional utilities: OpenSSL CLI (certificate inspection), curl/Postman (API validation), and a modern Chromium-based browser for dashboard rendering.

5. Dependency Inventory

Backend libraries (from backend/requirements.txt):

Library	Role
fastapi	HTTP API framework and endpoint layer
uvicorn	ASGI server runtime
pydantic	Data contracts and request/response validation
python-multipart	Multipart/form-data request handling
requests	HTTP client calls for integrations
cryptography	Cryptographic processing and security operations
python-dotenv	Environment variable loading from .env
apscheduler	Scheduled and recurring scan jobs
dateparser	Natural-language date and schedule parsing
reportlab	Executive PDF report generation

Frontend runtime + build libraries (from package.json):

Category	Packages
Runtime	react, react-dom, react-router-dom, react-force-graph-2d
Build	vite, typescript, @vitejs/plugin-react
Styling	tailwindcss, postcss, autoprefixer, @tailwindcss/forms, @tailwindcss/container-queries
Lint/Quality	eslint, @eslint/js, typescript-eslint, eslint-plugin-react-hooks, eslint-plugin-react-refresh, globals
Type Definitions	@types/node, @types/react, @types/react-dom

6. Environment Configuration

Create a backend environment file at backend/.env and set only the keys required by the optional features you plan to enable (AI summary and/or email dispatch).

Variable	Required	Description
GEMINI_API_KEY	Optional (feature-dependent)	Enables AI summarization and chatbot-assisted responses
SMTP_EMAIL	Optional (feature-dependent)	Sender address used for scheduled/direct report emails
SMTP_PASSWORD	Optional (feature-dependent)	SMTP authentication secret for sender account

7. Installation Steps

A) Clone repository

```
git clone  
https://github.com/Mohitlikestocode/Quantum-Proof-Systems-Scanner_PNB.git cd  
Quantum-Proof-Systems-Scanner_PNB
```

B) Frontend dependency install

```
npm install
```

C) Backend dependency install

```
cd backend pip install -r requirements.txt
```

D) Run backend service

```
cd backend uvicorn main:app --reload --host 0.0.0.0 --port 8000
```

E) Run frontend service

```
npm run dev
```

F) Production frontend build (optional validation)

```
npm run build npm run preview
```

8. Runtime Verification Checklist

- Backend health: API service responds and docs open at <http://localhost:8000/docs>
- Frontend health: Vite serves dashboard on the local URL reported in terminal
- Functional validation: run at least one scan and confirm dashboard tables/graphs populate
- Reporting validation: generate a PDF report and verify file creation/download
- Optional integration check: validate AI summary or SMTP dispatch when keys are configured

9. Troubleshooting and Hardening

- Use Python virtual environments to isolate backend packages and prevent global version conflicts.
- Pin dependency versions for reproducible CI and deployment builds.
- Restrict environment file access and rotate API/SMTP secrets periodically.
- Run the backend behind a reverse proxy for TLS termination in shared environments.
- Enable periodic dependency audits (npm audit / pip-audit) in CI pipelines.

This installation document is comprehensive for current repository implementation and submission review readiness.